前置知識與目標

- 前置知識為 BST、旋轉概念、遞迴、機率加權期望、動態規劃與攤銷分析。
- 能分辨 optimal BST 的靜態加權目標、AVL 與紅黑樹的最壞高度保證，以及 splay tree 的序列保證。
- 能以中序序列及結構不變量檢查旋轉，而不是只背圖形。

最佳化什麼成本

- 高度最小化降低最壞搜尋深度，未必最小化平均搜尋成本。
- 若鍵 `k_i` 以已知機率 `p_i` 被查詢，成功搜尋期望比較數為 `Σp_i(depth(k_i)+1)`。
- 若失敗查詢也計成本，要給 n+1 個鍵間空隙機率 `q_0,…,q_n`，並定義 dummy leaf 的成本慣例。
- Optimal BST 適合分布相對固定的靜態索引，頻繁插刪時重新最佳化成本必須另外計算。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

成功查詢版 optimal BST 遞迴

- 定義 `C[i,j]` 為有序鍵區間 `[i,j]` 的最小加權比較成本，空區間成本為 0。
- 設 `W[i,j]=Σ_{k=i}^j p_k`。若選 r 為根，兩子樹每個鍵都多一次比較。
- 因此 `C[i,j]=W[i,j]+min_{r=i..j}(C[i,r-1]+C[r+1,j])`。
- 依區間長度由短到長計算，前綴和使 W 查詢 O(1)，時間 `O(n³)`、空間 `O(n²)`，並記錄 root 重建樹。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

加權例子

- 三鍵機率 `p=[0.1,0.2,0.7]`。若選第二鍵為根，成本為 `1+0.1+0.7=1.8`。
- 若選第三鍵為根，左子樹以第二鍵為根，成本為 `0.7·1+0.2·2+0.1·3=1.4`。
- 最低高度的根二不會得到最低期望成本。高頻鍵靠近根可補償低頻鍵更深的成本。
- 「每次選最高機率鍵」仍不保證全域最優。反例 `p=[0.4,0.35,0.25]` 選第一鍵為根的最小成本是 1.85，選第二鍵為根則是 1.65。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

含失敗查詢的標準 DP 慣例

- 令 `e[i,i-1]=q_{i-1}`，空子樹視為一次 dummy 比較。令 `w[i,j]=Σp_i..p_j+Σq_{i-1}..q_j`。
- 遞迴仍為 `e[i,j]=w[i,j]+min_r(e[i,r-1]+e[r+1,j])`，但此時不可以把空子樹成本錯設為 0。
- 此成本是 `Σp_i(depth(key_i)+1)+Σq_j(depth(dummy_j)+1)`。若只計真鍵比較，失敗部分各減一次，總差為常數 `Σq_j`，最優樹不變。
- Knuth 的 `O(n²)` 最佳化需要適用的非負權重與單調根性質。一般任意區間 DP 不能只加相同名稱就沿用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Optimal BST 的完整區間表與失敗成本數例

```
for i=1..n+1: C[i,i-1]=0
for length=1..n:
    for i=1..n-length+1:
        j=i+length-1; C[i,j]=infinity
        for r=i..j:
            candidate=W(i,j)+C[i,r-1]+C[r+1,j]
            if candidate<C[i,j]: C[i,j]=candidate; root[i,j]=r
build(i,j):
    if i>j: return null
    r=root[i,j]; return Node(k[r],build(i,r-1),build(r+1,j))
```

成功版p=[.1,.2,.7]：C11=.1、C22=.2、C33=.7；C12候選根1=.5、根2=.4；C23候選根2=1.6、根3=1.1；C13候選根1=2.1、根2=1.8、根3=1.4。重建為3的左子2、2的左子1，三鍵深度2,1,0，直接加權=.1×3+.2×2+.7×1=1.4。

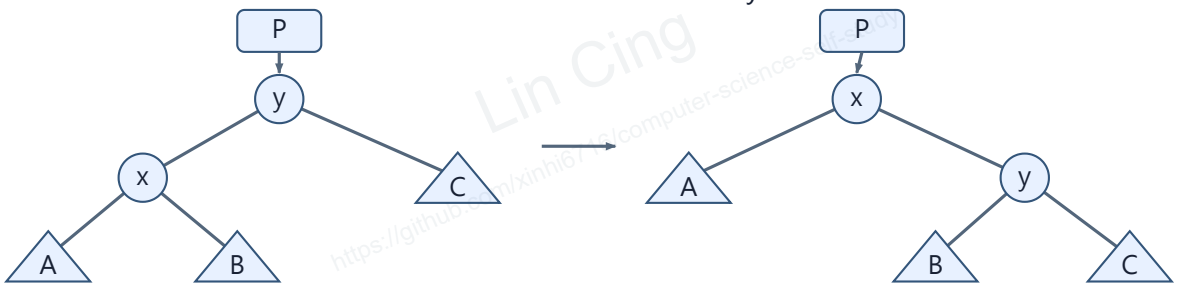
若p1=.6,q0=.1,q1=.3，只有一真鍵。dummy也計一次的初始化e10=.1,e21=.3,w11=1，得e11=1.4；只數真鍵比較則每次成功或失敗都比較該真鍵一次，期望1，兩者差q0+q1=.4。不能把e空區初始化0卻仍報dummy成本1.4。

最優子結構證明：固定根r後，左右鍵範圍被順序唯一切開，任一非最優子樹都可替換成同鍵範圍的更小加權成本樹，而整體根比較成本W不變，與最優矛盾。依區間長度歸納便證DP，不需貪婪取最高機率鍵。

旋轉保留鍵的中序次序

- 右旋前根 y、左子 x，子樹順序為 `A<x<B<y<C`。右旋後根 x、右子 y，中序順序仍相同。
- 指標更新依序保留 B，令 `y.left=B`、`x.right=y`，再接回原父。若維護 parent，B 與兩根的 parent 也要更新。
- 高度、子樹大小或其他 augmentation 先更新降到下面的 y，再更新升上來的 x。
- 旋轉只保證 BST 次序不變，不會自動維持任意額外不變量。紅黑色彩或 AVL 高度要各自修復。

右旋：外部父與 A < x < B < y < C



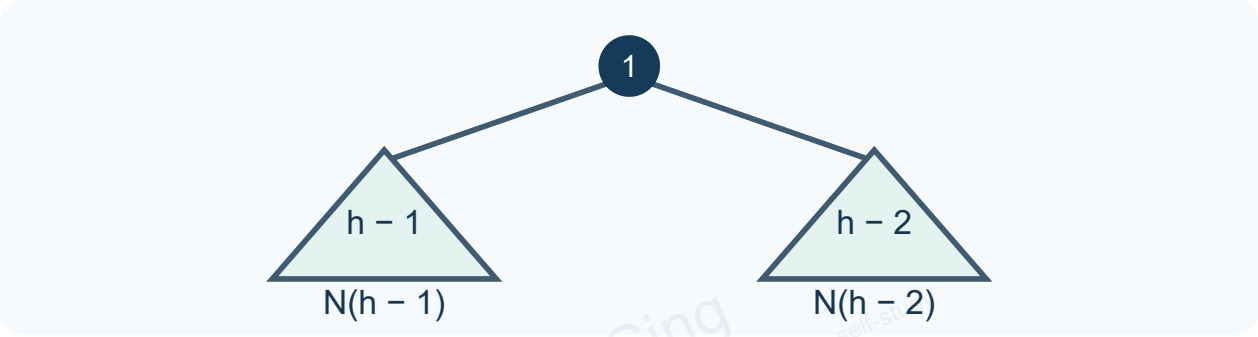
AVL 的局部高度約束

- `h(null)=-1`，`h(leaf)=0`。balance factor `bf(x)=h(left)-h(right)` 必須屬於 `{-1,0,1}`。
- 對 h≥1，高度 h 的最小點數 `N(h)=1+N(h-1)+N(h-2)`，基底 N(−1)=0、N(0)=1；因最高子樹高 h−1，另一側至少 h−2。
- 此遞迴以 Fibonacci 數增長，故 h=`O(log n)`。搜尋、插入與刪除都有最壞 `O(log n)` 路徑長。
- 只檢查根平衡不夠，約束必須對所有節點成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AVL 高度界：遞迴為何恰好成立？

令空樹高度為 −1、葉高度為 0，並以 N(h) 表示高度恰為 h 的 AVL 樹最少節點數。



高度為 h 的根至少有一側高 h−1；平衡條件要求另一側至少高 h−2。兩側各取對應最小樹即可達到下界，因此 N(h)=1+N(h−1)+N(h−2)，不是只有不等式。從 N(−1)=0、N(0)=1 出發，令 $F_0=0$、$F_1=1$，歸納得 $N(h)=F_{h+3}-1$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

用高度界推算 200 個節點

邊數高度 h	7	8	9	10
最少節點 N(h)	54	88	143	232

高度 10 至少要 232 個節點，因此 200 個節點的高度至多 9。此界可以達成：根的左右子樹均高 8，分別放 88 和 111 個節點，連同根恰好 200。高度 8 的最小樹有 88 個節點；由此樹以 AVL 插入增加 23 個鍵，在小於 N(9)=143 時高度不可能升至 9，因此可得到 111 個節點、高度仍為 8 的子樹。

另一慣例把葉高記為 1，所得數值為 10 層。比較答案時先核對高度是算「邊」還是算「節點層數」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AVL 插入的四種局部形狀

- LL 為 x 左高且新點在左子的左側，對 x 右旋。RR 對稱左旋。
- LR 先對左子左旋，再對 x 右旋。RL 對稱處理。
- 原例插入 `30,10,20`，形成 LR。先讓 20 升到 10 上方，再讓 20 升到 30 上方，結果根 20、左右 10、30。
- 插入修復在第一個失衡祖先旋轉後，該子樹高度回到插入前，通常不需再向上旋轉，但仍需正確更新 metadata。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AVL 刪除可能連續修復

- 先依 BST 刪除，從實際減少高度的位置往根更新。刪除後的局部修復不保證高度回到刪除前。
- 若 `bf(x)=2` 且 `bf(left)≥0`，右旋；若左子 bf<0，做 LR。右高情況對稱。
- 刪除特有的子 bf=0 情況不能沿用只看插入方向的判定。
- 修復最多沿根路徑進行 `O(log n)` 次局部更新，總最壞 `O(log n)`。只修第一個失衡點可能留下更高祖先失衡。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AVL 重算、插入與刪除的共同程序

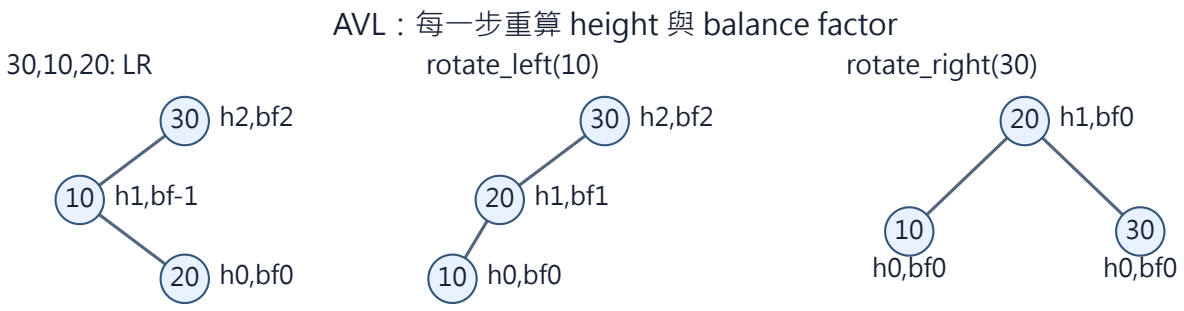
```
fix(x):
    if x==null: return null
    recompute_height(x)
    if bf(x)==2:
        if bf(x.left)<0: x.left=rotate_left(x.left)
        return rotate_right(x)
    if bf(x)==-2:
        if bf(x.right)>0: x.right=rotate_right(x.right)
        return rotate_left(x)
    return x
insert(x,k):
    if x==null: return leaf(k)
    if k<x.key: x.left=insert(x.left,k)
    else if k>x.key: x.right=insert(x.right,k)
```

https://github.com

AVL 重算、插入與刪除的共同程序 （續）

```
    else: update/reject duplicate
    return fix(x)
erase(x,k):
    if x==null: return null
    if k<x.key: x.left=erase(x.left,k)
    else if k>x.key: x.right=erase(x.right,k)
    else:
        if x has at most one child:
            child=the nonnull child or null; release(x); return child
        s=minimum(x.right); x.contents=copy(s.contents)
        x.right=erase(x.right,s.key)
    return fix(x)
```


AVL 重算、插入與刪除的共同程序 （續）



每個旋轉先重算下降節點，再重算新根；不能只更換root指標而沿用舊height。30,10,20的LR過程：旋前30(h2,bf2)、10(h1,bf−1)、20(h0,bf0)；先旋10後30仍(h2,bf2)，其左20(h1,bf1)；再旋30後20(h1,bf0)、兩葉高度0。

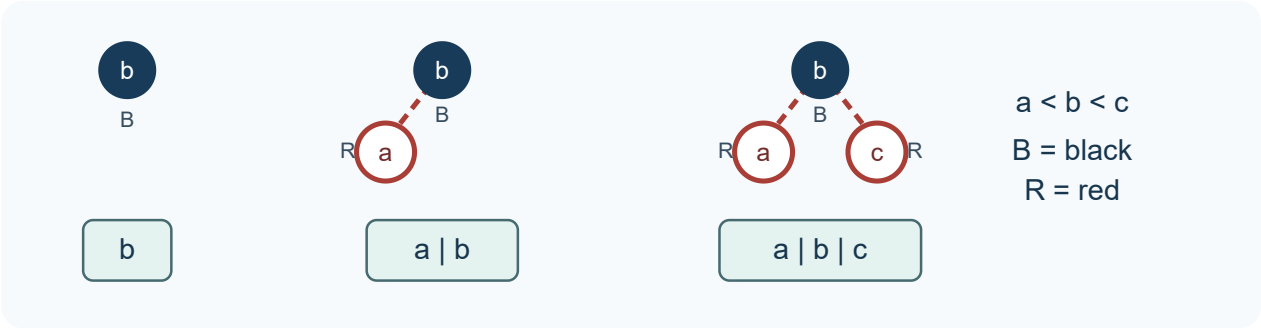
刪除例4的左2含1,3、右5為葉：刪5後4的bf2、左2的bf0，fix採單右旋而非LR，結果2左1右4、4左3。新局部高度仍2，所以此例上層無高度減少；其他刪除旋轉可使高度下降，需回溯繼續fix至根。遞迴每層常數旋轉與metadata工作，總O(log n)。

一般紅黑樹不變量

- 每個節點為紅或黑，根黑，所有 NIL 葉為黑。
- 紅節點的子節點必為黑，且從任一節點到各後代 NIL 路徑的黑節點數相同。
- 本章講一般 node-colored 紅黑樹，不要求所有紅連結左傾。Princeton 的 LLRB 是較強的另一套表示慣例。
- 檢驗圖時必須把隱含 NIL 算入黑高約定，不能只目測有色實節點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

紅黑樹與 2–3–4 樹：把紅連結收進父節點

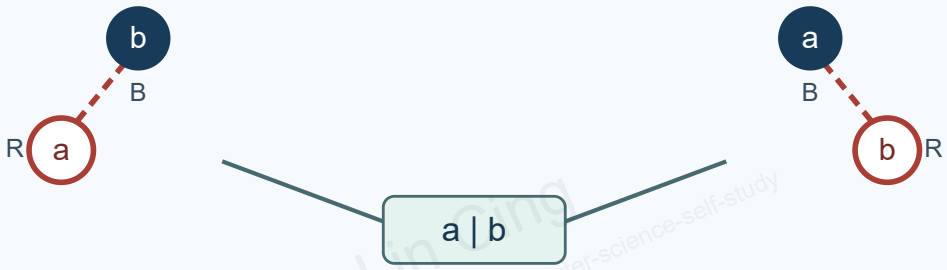


把每個紅節點與其黑色父節點合成一個多鍵節點。沒有紅子節點時是 2-node；一個紅子是 3-node；兩個紅子是 4-node。圖中深色實心節點標 B，空心節點標 R，紅子連結另以虛線表示，列印成灰階仍能辨識。

紅節點不能再有紅子節點，故每組最多三個鍵；沿路每跨一個合併節點就跨一個黑節點。所有根到 NIL 路徑黑高相同，對應的多路樹外部葉深度也相同。原來的 BST 中序順序決定各子樹的鍵值區間。

一棵 2–3–4 樹可有多種紅黑表示

含鍵 a<b 的 3-node，既可由黑 b 帶紅左子 a 表示，也可由黑 a 帶紅右子 b 表示；兩者合併後相同。一般紅黑樹允許兩種方向，左傾紅黑樹（LLRB）則採額外方向約束，其具體允許的暫時狀態仍須看實作。



AVL 的 LR 或 RL 修復由兩次基本單旋組成。有些說法把整個修復稱為一次雙旋，有些則計作兩次旋轉。報告旋轉次數時要說清楚計數單位；不論哪種說法，中間樹都必須保持 BST 次序。

紅黑樹高度上界

- 定義 bh(x) 為 x 以下、不含 x、含 NIL 的路徑黑點數。對 bh=b 的子樹，實節點數至少 `2^b-1`。
- 節點數下界可歸納證明：兩個子樹的黑高都至少 $b-1$，各至少有 $2^{b-1}-1$ 個實節點，加上根即至少 2^b-1；NIL 為零節點基底。紅點不能連續，因此任何根到 NIL 路徑至多有約兩倍黑高的邊數。
- 由 `n≥2^{bh(root)}-1` 得高度不超過 $2\log_2(n+1)$ 的標準界，端點常數依高度約定處理。
- 這是最壞高度保證，不需要鍵隨機或查詢分布均勻。

紅黑插入修復的情況

- 新葉設紅，可保留原路徑黑高。若父黑即可停止，若父紅則產生 red-red。
- 叔叔紅時，把父與叔染黑、祖父染紅，將問題上推到祖父。
- 叔叔黑時，若是折線先旋父變直線，再旋祖父並交換適當顏色。鏡像情況對稱。
- 最後把根染黑。原創插入 `10,5,1` 會對 10 右旋，得到黑根 5、紅子 1 與 10。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

紅黑插入修復的完整祖先迴圈

```
insert_fix(z):
    while color(z.parent)==red:
        p=z.parent; g=p.parent
        if p==g.left:
            u=g.right
            if color(u)==red:
                p.black; u.black; g.red; z=g
            else:
                if z==p.right:
                    z=p; rotate_left(z); p=z.parent; g=p.parent
                p.black; g.red; rotate_right(g)
        else: apply exact left/right mirror
    root.black
```

新資料葉為紅、兩子NIL黑；根parent使用黑色哨兵，使空parent不被解參照。10先成黑根；插5紅不修；插1紅造成父5紅叔NIL黑，直線旋10右，得5黑、1與10紅。插7到10左，父10紅叔1紅，兩者黑、祖父5紅再根黑。最後每條根到NIL三黑，中序1,5,7,10。

叔紅情況只上推z，沒有保證一次結束；叔黑折線先旋父再重取p/g，否則第二次旋轉用到舊parent會接錯。每輪問題上升或常數旋轉結束，總O(log n)時間。色彩由一般紅黑不變量推導，不要混用LLRB強制左傾的程式分支。

紅黑刪除的黑高缺額

- 若刪掉紅節點，不改變黑高。若黑節點被紅子取代，把取代子染黑即可。
- 其餘情況在取代位置 x 追蹤「少一個黑」的缺額。這是分析狀態，不必真的新增第三種節點顏色。
- 設 x 是左子、兄弟為 w。若 w 紅，旋轉並換色，使兄弟轉為黑再處理。
- 若 w 黑且兩子黑，把 w 染紅並將缺額推到父。若父原紅，把父染黑即可消除缺額。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

紅黑刪除的近侄與遠侄

- 仍假設 x 在左側。w 的左子是近侄、右子是遠侄。
- 若 w 黑、遠侄黑、近侄紅，先右旋 w 並換色，化為遠侄紅的情況。
- 若遠侄紅，令 w 取得父的原顏色，父與遠侄染黑，對父左旋，消除 x 側黑高缺額。
- x 在右側時完全鏡像。每次修復只做常數局部工作或將問題上推，總最壞 `O(log n)`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

旋轉與紅黑刪除的移植入口

```
right_rotate(y):
    x=y.left; require x!=NIL
    b=x.right; p=y.parent
    y.left=b
    if b!=NIL: b.parent=y
    x.parent=p
    if p==NIL: root=x
    else if p.left==y: p.left=x
    else: p.right=x
    x.right=y; y.parent=x
    pull(y); pull(x)
transplant(u,v):
    if u.parent==NIL: root=v
    else if u==u.parent.left: u.parent.left=v
    else: u.parent.right=v
    v.parent=u.parent       // including NIL: deletion needs its deficit parent
RB_erase(z):
```


旋轉與紅黑刪除的移植入口 （續）

```
    y=z; removed_color=y.color
    if z.left==NIL:
        x=z.right; p=z.parent; transplant(z,x); pull_up(p)
    else if z.right==NIL:
        x=z.left; p=z.parent; transplant(z,x); pull_up(p)
    else:
        y=minimum(z.right); removed_color=y.color; x=y.right
        if y.parent==z: x.parent=y
        else:
            p_old=y.parent; transplant(y,x)
            pull_up(p_old)          // repair old path before y changes parent
            y.right=z.right; y.right.parent=y
        transplant(z,y); y.left=z.left; y.left.parent=y; y.color=z.color
        pull(y); pull_up(y.parent)  // repair y at its new position
    release(z)
    if removed_color==black: delete_fix(x)
```

此頁採一般紅黑樹的黑色NIL哨兵，非資料節點。root.parent=NIL，NIL的color永遠黑；delete_fix採〈紅黑刪除四情況的完整控制流程〉。左旋是左右完全互換的程序。本節的完整增廣為size：NIL.size=0、pull(x)=1+size(left)+size(right)；pull_up由下向上重算至NIL，純紅黑色彩不需因旋轉自動改色。旋轉只在真子存在時執行，且先保留b與外部父p，才更改連結。

旋轉與紅黑刪除的移植入口 （續）

兩子刪除移動後繼節點y到z的位置，不複製鍵後直接釋放y。決定是否少一黑的是y原位置的removed_color，不是移到新位置後取得的z.color。x是y原位置的唯一右子或NIL。y緊鄰z時，x.parent必直接設y；否則先把y的右子移植到舊位置，再把整個z右子樹接y。最後左子樹與原z顏色接回，釋放的是z，其他元素身分保留。

NIL可由所有空子共用，但修復當前缺額時必須保留該x的parent。旋轉不替任意空中間子b重寫NIL.parent，以免覆蓋缺額位置；只有明確的transplant／緊鄰後繼分支設定它。刪唯一黑根使root=NIL，delete_fix迴圈因x==root跳過，仍是合法空樹。

正確性分工：BST移植保留中序與所有未刪元素，複製z顏色使y新位置的黑數不變，唯一可能缺額在y舊位置。四情況迴圈只處理這個缺額，維持其他子樹合法，直到在紅父、遠紅侄或根處消除。搜尋、後繼與修復皆走O(log n)路徑，移植／旋轉每次常數指標工作；另外兩條pull_up路徑各長O(log n)，不能省略其metadata更新。

子樹大小、選秩與兩條修復路徑

```
pull(x):
    require x!=NIL
    x.size=1+x.left.size+x.right.size
pull_up(x):
    while x!=NIL: pull(x); x=x.parent
select(k):
    require 1<=k<=root.size          // one-based rank
    x=root
    while x!=NIL:
        r=x.left.size+1
        if k==r: return x
        if k<r: x=x.left
        else: k-=r; x=x.right
    unreachable                     // sizes and rank invariant forbid this
```

每個資料節點的不變量為 size(x)=1+size(x.left)+size(x.right)，NIL.size=0。新資料葉size=1；插入先完成BST連接、從新葉的父pull_up至根，再做insert_fix。每次旋轉都先pull下降節點，再pull新局部根，才能在修復前後維持精確size。

子樹大小、選秩與兩條修復路徑 （續）

以合法插入4,2,6,1,3,5,7後刪紅葉1為例，刪除不需顏色修復或旋轉，但size仍須改變：

欄位	刪除前	只transplant、漏pull	正確刪除後
size(2)	3	3	2
size(4)	7	7	6
select(3)	節點3	走到NIL	節點4

漏更新時，根4讀到左size=3，誤認自己的秩為4，令第3秩往左。節點2再把它導向3的右NIL。正確左size=2時，根本身就是第3秩，直接回4。顏色與黑高正確不會自動證明增廣欄位正確。

至多一子分支只改z原父以上的子集合，故transplant後從保存的父p向上pull。兩子且後繼y不緊鄰z時，先在y原位置接回x，立刻從p_old向上pull；此時原祖先鏈仍完整。再把y搬到z位置，接齊左右子後pull(y)，並由其新父向上pull。緊鄰後繼省略原位置鏈，仍要做新位置鏈。兩鏈可能相交，但只多常數倍對數工作。

一般正確性由下向上歸納：鏈外子樹未變，最底變動節點的兩子size已正確，pull便算對該點，逐層到根。後續紅黑修復只改色或旋轉；改色不影響size，旋轉僅改兩個局部根的子集合。依下降點→上升點pull後，局部總元素集合不變，其外部祖先的size不需再改。select的迴圈保留「k是目前子樹內所求的有效秩」，因左size正確而只排除不可能區間，有限高度保證終止。

子樹大小、選秩與兩條修復路徑 （續）

size版本的兩條修復路徑與常數次旋轉，連同搜尋／後繼均為O(log n)最壞時間；select亦為O(log n)。若改維護height，旋轉後的局部總高度可能改變，必須另向祖先傳播，不能沿用size的「局部元素數不變」理由。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

紅黑刪除四情況的完整控制流程

```
while x!=root and color(x)==black:
    if x==x.parent.left:
        p=x.parent; w=p.right
        if color(w)==red:
            w.black; p.red; rotate_left(p); w=p.right
        if color(w.left)==black and color(w.right)==black:
            w.red; x=p
        else:
            if color(w.right)==black:
                w.left.black; w.red; rotate_right(w); w=p.right
            w.color=p.color; p.black; w.right.black
            rotate_left(p); x=root
    else: apply exact left/right mirror
x.black
```

前提是BST刪除／節點transplant已完成，x代表少一黑的位置，包含NIL哨兵且可取得其parent；原被實際移除節點若是紅色不需此迴圈。`.black/.red` 表示設色，不是新建節點。每次旋轉須更新外部parent及root。

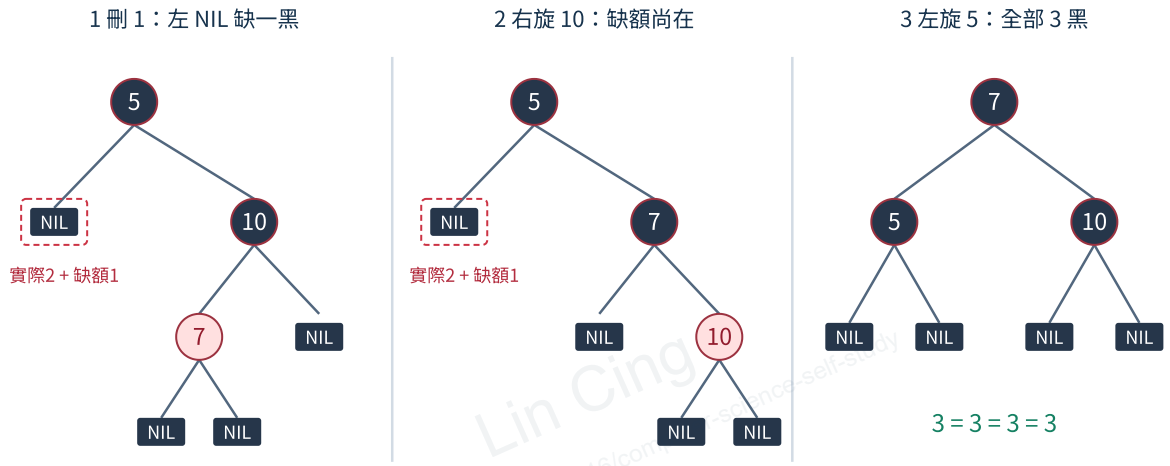
紅黑刪除四情況的完整控制流程 （續）

具體合法樹：根5黑，左1黑，右10黑，10左7紅，其餘NIL黑。刪1後左NIL缺一黑，兄弟10黑、近侄7紅、遠侄NIL黑。先把7黑、10紅，右旋10，得兄弟7黑且遠侄10紅。再令7取父5原黑、5黑、10黑，左旋5，得黑根7、黑子5與10。每條根到NIL共3黑，恢復刪前對外黑高；中序從[1,5,7,10]變[5,7,10]。

若兄弟紅，第一次轉換不立即消除缺額，只把兄弟轉黑後再進入其他情況；若兄弟兩子都黑，缺額向上，可能一直到根。不能把四個if任意改成互斥早退，也不能只修單層。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

紅黑刪除的缺額與兩次旋轉對照



B：黑；R：紅；所有 NIL 皆黑。中序保持 5,7,10。

同一合法起始樹 5B(1B,10B(7R,NIL)) 刪除 1。圖的 NIL 都是黑；紅框虛線只表示左側分析上的一黑缺額。第一態左路實際含 5B、NIL 共 2 黑，右路為 3 黑；把缺額記入才暫時相等。第二態先把近侄 7 染黑、10 染紅並右旋 10，缺額仍在 5 的左 NIL，尚未修完。第三態 7 取 5 的黑色、5 與遠侄 10 染黑，再左旋 5，全部四條根到 NIL 路徑各 3 黑，缺額消失。

紅黑刪除的缺額與兩次旋轉對照 （續）

旋轉前後中序從 [5,7,10] 保持不變。每態的外部根連結與 parent 都要同步更新；圖中的黑數是含局部根及 NIL 的路徑計數，與前文 bh(x) 排除 x 的定義分開使用。若只做第二態的右旋便停止，左路仍少黑，不能把「近侄轉成遠侄」當成修復完成。

Splay 的 zig、zig-zig、zig-zag

- 查到 x 後把 x 旋到根。若只剩父而無祖父，做一次 zig。
- 若 x 與父同為左子或同為右子，做 zig-zig，先旋父相對祖父，再旋 x 相對父。
- 若方向相反，做 zig-zag，先旋 x 相對父，再旋 x 相對祖父。
- 這些步驟保留 BST 中序，但不維持固定平衡因子，也不保證每次操作後高度為 $O(\log n)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Splay 的存取引理與攤銷

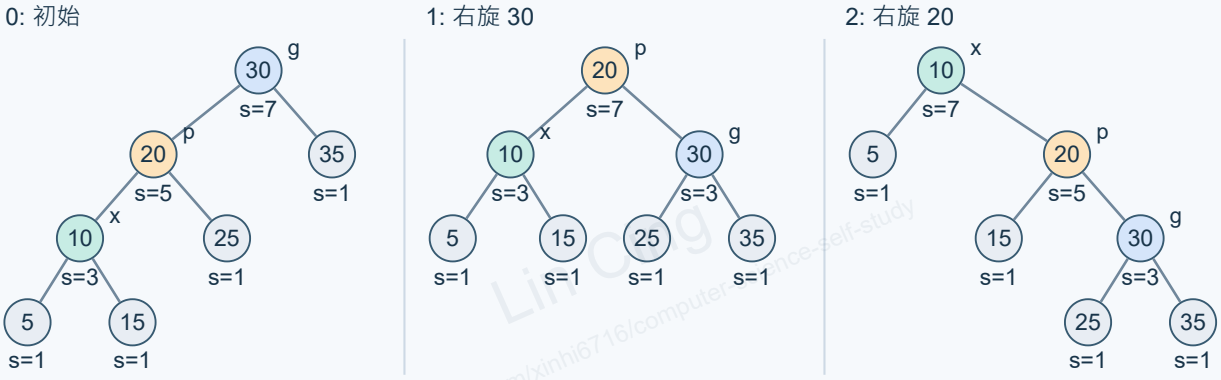
- 對每點設定權重，令子樹權重和為 $s(x)$，rank $r(x)=\log_2 s(x)$，勢能 $\Phi=\Sigma r(x)$。
- access lemma 將一次 splay 的攤銷成本界為 $O(1+\log(s(root)/s(x)))$。
- 等權時得到每次操作攤銷 $O(\log n)$。對任意起始樹要保留初始勢能項，不能把任意單次操作說成最壞 log n。
- 原創鏈 $1\to2\to3\to4$ 搜尋 4，實際可走 4 點；反覆存取已在根的 4 卻只需常數工作，反映序列自調整。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Zig-zig：先讓父節點上升

固定 x=10、p=20、g=30。橘色 p 先越過 g，再讓綠色 x 越過 p；三個字母代表同一批節點，旋轉後不重新命名。

zig-zig：先旋 p，再旋 x



互斥集合：舊 x = {5,10,15}；新 g = {25,30,35}。
單位權重；s 含節點自己。三態中序皆為 5,10,15,20,25,30,35。

中間態根是 20，最後根才是 10。若先讓 x 越過 p，再越過 g，會得到不同樹形；「把目標轉到根」不足以指定 splay 的操作順序。

Zig-zig 的勢能帳，可以逐項核對

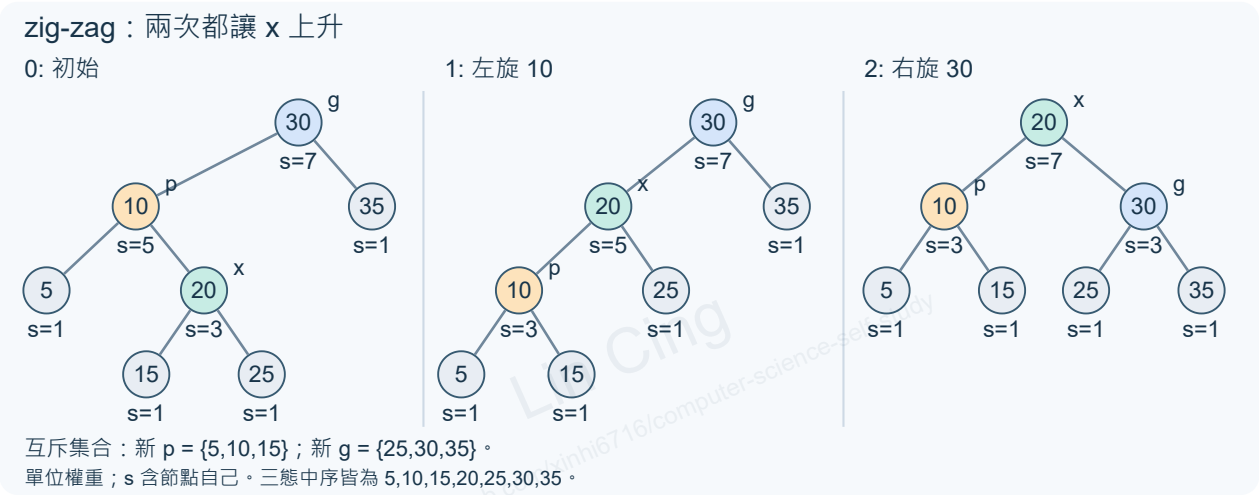
每個節點權重為 1，$s(v)$ 是含自己的子樹點數，$\Phi = \sum_v \log_2 s(v)$。四個葉節點的 rank 都是 0，只列會改變的三點。

節點	旋轉前 s	右旋 30 後 s	再右旋 20 後 s
x=10	3	3	7
p=20	5	7	5
g=30	7	3	3

前後勢能都為 $\log_2(3 \cdot 5 \cdot 7) = \log_2 105$，故 $\Delta\Phi = 0$，兩次旋轉的攤銷成本 $\hat{c} = 2$。引理給的上界為 $3\log_2(7/3) \approx 3.667$，確實大於 2。**勢能不必每次下降；它要支付的是整段操作的總成本。**

Zig-zag：兩次都讓目標上升

固定 x=20、p=10、g=30。先左旋 10，再右旋 30；四個葉節點的連結也必須隨旋轉接回。



最後 x 的左右子樹分別以 p、g 為根，兩者互不重疊。這正是下一頁使用凹性不等式時，要找出的兩個集合。

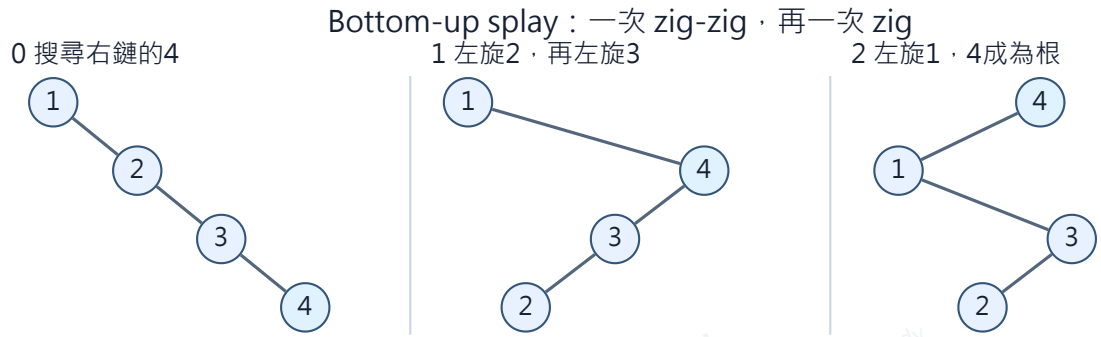
Zig-zag：數值如何對回凹性不等式？

節點	旋轉前 s	左旋 10 後 s	再右旋 30 後 s
x=20	3	5	7
p=10	5	3	3
g=30	7	7	3

$\Delta\Phi = \log_2(7\cdot 3\cdot 3) - \log_2(3\cdot 5\cdot 7) = \log_2(3/5) \approx -0.737$。所以 $\hat{c} = 2 + \Delta\Phi \approx 1.263$，小於 $2\log_2(7/3) \approx 2.445$。

圖中的兩個新子樹大小 a=b=3，且 a+b=6≤S=7。由 $(a-b)^2 \geq 0$ 可得 $ab \leq ((a+b)/2)^2 \leq (S/2)^2$，取 $\log_2$ 得 $\log_2 a + \log_2 b \leq 2\log_2 S - 2$。正權重確保可取對數；**本例檢查數值，一般證明使用的是任意正 a、b。**

從右鏈搜尋到完整 splay



搜尋依序訪問 1、2、3、4；伸展先左旋 2、左旋 3，再左旋 1。最後 4.left=1、1.right=3、3.left=2，中序仍為 [1,2,3,4]。再次搜尋 4 只訪問根、不旋轉。這組操作前後的勢能同為 $\log_2 24$，此例仍符合存取引理；攤銷保證不要求每次勢能下降。

Access lemma 的局部記帳

固定每個節點的正權重，令 $s(v)$ 為子樹權重和、$r(v)=\log_2 s(v)$，勢能 $\Phi=\sum_v r(v)$。每次旋轉收取 1 單位成本；沒有旋轉的存取仍有常數搜尋成本。

一次雙旋只改變 x、p、g 的子樹權重。撇號表示雙旋完成後，由 $r'(x)=r(g)$ 消去一項，可得 $\hat{c}=2+\Delta\Phi=2+r'(p)+r'(g)-r(x)-r(p)$。

若兩個互不重疊的子樹權重為 a、b，且 a+b≤S，前頁已證明 $\log_2 a+\log_2 b\le 2\log_2 S-2$。接下來依雙旋形狀，選出這兩棵子樹。此處計算旋轉數；搜尋深度與旋轉數相差常數，指標操作的固定倍數可透過整體縮放勢能支付。

Zig-zig 的攤銷不等式

舊 x 子樹與新 g 子樹互不重疊，且都包含在新 x 子樹。以 a=s(x)、b=s′(g)、S=s′(x) 代入，可得

$r(x) + r'(g) \le 2r'(x) - 2$。

又因 $r(p) \ge r(x)$、$r'(p) \le r'(x)$，所以

$\hat{c} \le 2 + r'(x) + r'(g) - 2r(x)$。

再用 $r'(g) \le 2r'(x) - 2 - r(x)$ 代入上式，整理得到 $\hat{c} \le 3(r'(x) - r(x))$。能相加的是 x 的 rank 增量，不是每一步的樹高下降量。兩次旋轉本身並不代表常數攤銷成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Zig-zag、zig 與望遠鏡和

Zig-zag。 新 p 與新 g 子樹互不重疊，因此 $r'(p) + r'(g) \le 2r'(x) - 2$。配合 $r(p) \ge r(x)$，得 $\hat{c} \le 2(r'(x) - r(x))$，亦不大於三倍差量。

最後一個 zig。 單旋的攤銷成本為 $1 + r'(p) - r(x) \le 1 + r'(x) - r(x)$，至多 $1 + 3(r'(x) - r(x))$。每次完整 splay 至多出現一次 zig。

整次存取。 各步的 x rank 差量相消，最後 x 成為根。令整樹權重為 W、存取前 x 子樹權重為 $s_0(x)$，則攤銷旋轉數至多

$3(\log_2 W - \log_2 s_0(x)) + 1$。

單位權重時 W=n、$s_0(x) \ge 1$，得到 O(log n) 攤銷界。總實際成本仍須加上初始勢能、減去最終勢能，故不排除某次存取花 Θ(n) 時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Splay 插入、刪除與適用限制

- 插入可先 BST 插入新葉，再把新葉 splay 至根。
- 刪除先把目標 splay 到根，移除後得到 L、R。若 L 非空，把 L 的最大鍵 splay 成根，其右子為空，可接上 R。
- 不成功搜尋可 splay 最後訪問的節點，維持對查詢序列的調整效果。
- splay 會在讀取時修改結構，並發唯讀存取與快照需求須額外設計。單次低延遲需求較適合有最壞界的平衡樹。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Splay 的完整旋轉與 split/join 更新

```
splay(x):
    while x.parent!=null:
        p=x.parent; g=p.parent
        if g==null: rotate x over p
        else if (x is p.left)==(p is g.left):
            rotate p over g; rotate x over p
        else: rotate x over p; rotate x over g
    root=x
insert(k):
    if root==null: root=Node(k,parent=null); return
    search k; splay(found or last visited node)
    if root.key==k: update/reject duplicate; return
    if k<root.key:
        z=Node(k,left=root.left,right=root,parent=null); root.left=null
        if z.left!=null: z.left.parent=z
```

Splay 的完整旋轉與 split/join 更新 （續）

```
    else:
        z=Node(k,left=root,right=root.right,parent=null); root.right=null
        if z.right!=null: z.right.parent=z
    root.parent=z; root=z
delete(k):
    search and splay target or last visited node; if absent: return
    (L,R)=(root.left,root.right)
    if L!=null: L.parent=null
    if R!=null: R.parent=null
    release root
    if L==null: root=R
    else:
        root=L; splay(maximum(L)); root.right=R
        if R!=null: R.parent=root
```

空樹insert直接建葉；split/join每次同時接回child與parent。刪根後先把L、R的parent清空，再釋放舊根，避免splay沿已釋放祖先走訪。鏈1→2→3→4搜尋4，先在祖父2與父3做兩個左旋，局部4左3左2；再對根1做zig，成4左1、1右3、3左2，並非單純變成反向鏈。

Splay 的完整旋轉與 split/join 更新 （續）

查詢的access lemma已逐旋轉證明；更新另有權重及勢能變化。單位權重插入split時，新根rank至多log(n+1)，被截斷子樹只會降低既有節點rank，故額外勢能增加O(log(n+1))。刪除後split不增rank，對L最大值splay用lemma，再接R只有新根rank最多增加log n，故更新也有O(log n)攤銷界。這不提供單次最壞高度或唯讀操作不修改的保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

比較與課程延伸

結構	最佳化對象	搜尋保證	更新代價特徵
Optimal BST	已知靜態加權成本	依分布最優	重建另計
AVL	局部高度差	O(log n) 最壞	維護高度，刪除可連續旋轉
紅黑樹	黑高與紅節點限制	O(log n) 最壞	色彩與旋轉修復
Splay	整段操作序列	O(log n) 攤銷	查詢本身會旋轉

研究所延伸可從 access lemma 改用非均勻固定權重推導存取頻率的代價，或驗證 optimal BST 的 root 單調性。前者需說明權重與查詢次數的關係，後者完整最佳化證明不在本稿範圍。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

學習檢查

- 為每個結構先寫清楚不變量、操作前提與成本單位。
- AVL 的高度是最壞情況；splay 的界限描述整段操作序列。
- 繪製旋轉時，同時檢查中序、父子指標與必要 metadata。

參考資料與署名

Lin Cing，《資料結構》手寫筆記。 Sleator、Tarjan，〈Self-Adjusting Binary Search Trees〉，JACM 32(3)，1985， §2：bottom-up splay 與 access lemma。圖中數值例採本章自訂鍵值與單位權重。 AVL 的課程定位核對 MIT 6.006 2020 L7。紅黑樹延伸對照 Princeton balanced search trees，該頁採 LLRB，不能混用插刪程式。 本章採用的比較成本、黑高及旋轉慣例均在相關段落定義；對照其他教材時，先確認端點與成本定義一致。

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study